

StarryOS 内核开发与 AI 辅助开发流水线

BigLab B 最终答辩

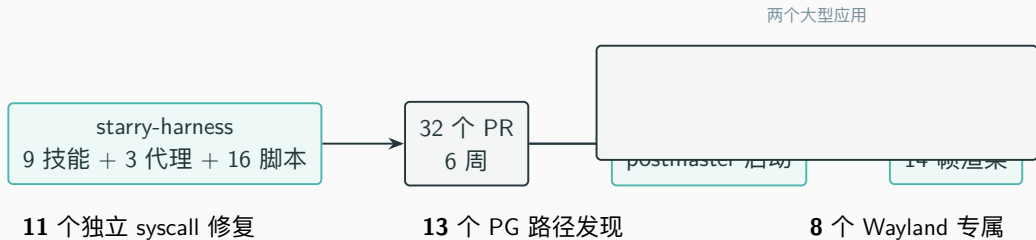
Joseph Joshua (jsph273)

2026-05-30

`rcore-os/tgoskits`

工作概览

工作总览



时间线: 2026-04-15 → 05-21, 六周

核心命题: 使 StarryOS 的 Linux syscall 语义从“基本可用”达到“精确对齐”。

实验 1: AI 开发流水线

动机：三个痛点

反馈循环过慢

build → rootfs → QEMU →
test
3-5 分钟/迭代

Linux 对照成本高

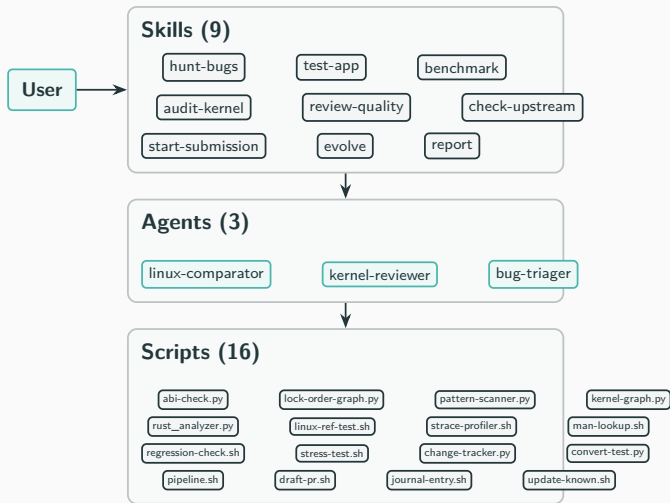
手动编译、手动运行
手动读 man page
逐字段人工对比

重复工作线性增长

每个 bug 的修复流程高度
相似
状态追踪与报告维护负担
随修复数量线性增加

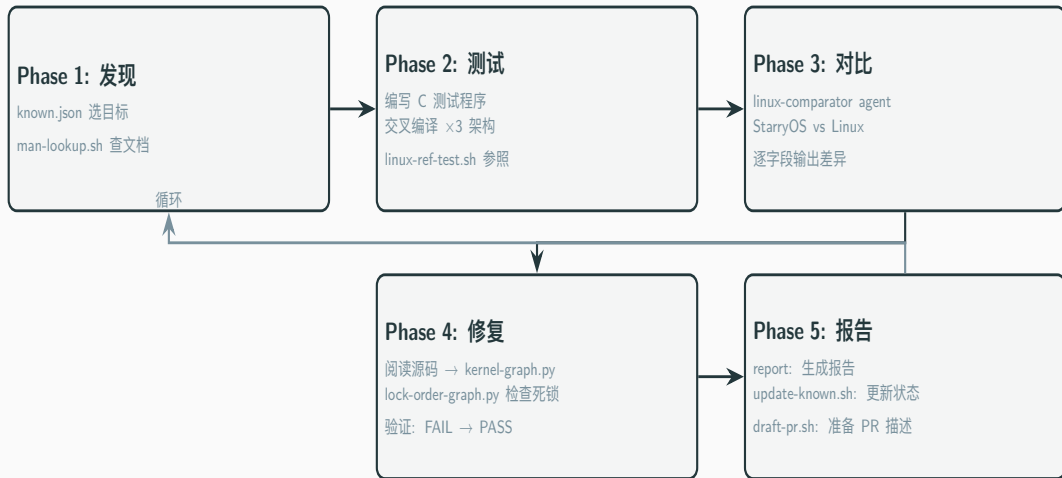
[starry-harness](#) 将这三类重复工作自动化。开发者保留架构决策与语义判断的控制权。

流水线架构



脚本是确定性的（同一输入永远同一输出）；代理处理需要判断的工作。

核心 workflow: hunt-bugs 五阶段



三道工程护栏

护栏	机制	效果
结构化输出	所有 Agent 输出经 JSON Schema 验证	下游可无歧义消费，状态变化可追踪
状态记忆	known.json 为所有缺陷的单一事实来源	防止重复修复、重复提交
审查否决权	kernel-reviewer 独立运行，只找问题不批准	不合格的修复退回重改，直至全部维度通过

审查维度：行为精确匹配 Linux？TOCTOU / UAF / 未初始化内存？锁顺序一致？死锁可能？错误路径回滚？回归测试覆盖所有触发条件？

实验 2: 系统调用修复

概述与 PR 总表

PR	日期	标题	子系统
#197	04-15	fix preadv/pwritev/preadv2/pwritev2 ABI	FS
#205	05-07	feat: implement mremap	Memory
#207	04-25	fix sigaltstack MINSIGSTKSZ	Signal
#208	04-19	fix getgroups, clock_gettime	Task/Timer
#209	04-30	fix ftruncate/pwrite64 negative values	FS
#210	04-23	fix getrandom flags validation	Misc
#211	04-24	fix copy_file_range validation	FS
#250	04-18	fix epoll_pwait sigsetsize	Signal/epoll
#318	04-23	fix loongarch64 QEMU memory	Arch
#503	05-16	feat: timerfd	Timer
#505	05-16	feat: crash dump on fatal signals	Signal

深挖三个代表性 PR: [timerfd](#) (fd 抽象层)、[preadv/pwritev](#) (syscall ABI 层)、[mremap](#) (内存管理层)。

Deep Dive: timerfd (#503): 动机与设计

timerfd 将 POSIX 定时器暴露为可被 epoll/select/poll 监听的文件描述符。read() 返回到期次数。systemd、dbus、Wayland event loop 均依赖此机制。

从零实现一个完整 FileLike 类型:

创建

```
timerfd_create(clockid, flags) → fd  
→ TimerFd { clock, interval,  
             expiration, is_one_shot }  
→ 注册到 FD_TABLE (FileLike trait)
```

设置

```
timerfd_settime(fd, flags,  
                new_value, old_value)  
→ itimerspec: {it_interval, it_value}  
→ TFD_TIMER_ABSTIME (绝对/相对)  
→ 更新底层 axtask 定时器
```

read(fd) → u64 (到期次数): 非阻塞空队列返回 EAGAIN; 读取后清零内部计数器。

timerfd (#503): 实现要点

FileLike trait 的完整实现:

```
fn read(&self) -> Result<u64> {  
    // 阻塞或非阻塞等待  
    // 返回到期次数  
    // 非阻塞空队列 → EAGAIN  
}  
fn write(&self) -> Err(EINVAL) {  
    // 不支持写操作  
}
```

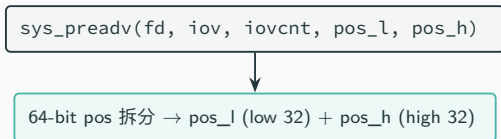
```
fn close(&self) {  
    // 取消底层定时器, 释放资源  
}  
fn poll(&self) -> EPOLLIN {  
    // 到期时 waker 被唤醒  
    // epoll 报告 EPOLLIN  
}
```

关键细节:

- `it_interval = {0, 0}`: 一次性定时器, 到期后不复用
- `it_interval > 0`: 周期性定时器, 到期后自动重新武装
- 时钟源: `CLOCK_REALTIME` (墙上时间)、`CLOCK_MONOTONIC` (单调时间)
- `epoll` 集成: `FileLike::poll()` 返回 `EPOLLIN`, 定时器到期唤醒 `epoll_wait`

Deep Dive: preadv/pwritev ABI (#197)

64 位文件偏移在 32 位寄存器 ABI 中被拆分为两个参数:



Linux 内核定义:

- `preadv / pwritev` → `SYSCALL_DEFINE5 (fd, vec, vlen, pos_l, pos_h)`
- `preadv2 / pwritev2` → `SYSCALL_DEFINE6 (fd, vec, vlen, pos_l, pos_h, flags)`

StarryOS 此前遗漏 `pos_h` 参数。更隐蔽的问题: `sys\_pwritev2` 从 `sys\_preadv2` 复制而来, 内部调用仍为 `.read_at()` 而非 `.write_at()`。此为 [copy-paste 产生实现错误的典型案例](#)。

修复: 重组 64 位偏移; 修正 `pwritev2` 内部调用方向。此 PR 后新增 `abi-check.py` 脚本, 自动比对每个 `syscall` 的参数数量与 Linux `SYSCALL_DEFINE`n 宏的元数。

Deep Dive: mremap (#205): 动机

`mremap` 在虚拟地址空间中重新映射已有的内存区域。glibc 的 `realloc` 对大块内存使用 `mremap` 而非 `malloc + memcpy + free`。

模式	标志	行为
原地扩展	无 <code>MREMAP_MAYMOVE</code>	尝试扩大 VMA，地址不变；失败返回 <code>ENOMEM</code>
可移动扩展	<code>MREMAP_MAYMOVE</code>	当前地址不够时寻找新地址，更新页表映射
固定地址	<code>MREMAP_FIXED</code>	指定目标地址，替换已有映射

此 PR 是理解“内核如何管理进程地址空间”的入口：涉及 VMA 操作、页表更新、地址对齐约束。

mremap (#205): 实现要点

```
mremap(old_addr, old_size,  
       new_size, flags, new_addr)
```

1. 验证 old_addr 页对齐, old_size > 0
2. 在 AddrSpace 中查找对应 VMA
3. 检查 VMA 精确匹配 old_size
4. 三种路径:
 - a. new_size <= old_size: 收缩 VMA
 - b. 原地可扩展: 直接扩大 VMA
 - c. MREMAP_MAYMOVE: 寻找空闲地址范围, 复制页表映射
5. 更新 VMA 集合与页表

关键约束:

- 新地址范围必须完全空闲
- MREMAP_FIXED 替换目标地址已有映射
- 收缩操作: 先 munmap 尾部页面, 再更新 VMA size
- 所有操作在 AddrSpace 锁保护下原子完成

与 mmap、munmap、brk 共同构成 StarryOS 虚拟内存管理的四个基础原语。

11 个 PR 中反复出现三类缺陷模式:

模式	案例	特征
遗留实现与新语义混淆	#197 preadv/pwritev	从早期简化实现演进时遗漏参数, copy-paste 未核验
直觉驱动边界处理	#207 sigaltstack、#250 epoll sigsetsize	Linux 的精确边界是 20 年 bug 修复的结果, 非直觉可替代
参数校验的静默缺失	#208 getgroups、#210 getrandom、#211 copy_file_range	基本路径可通, 非法参数不返回 EINVAL, 调用方误以为操作成功

从独立修复到大型应用

这些独立修复使 StarryOS 通过大部分单次 syscall 测试。

但真实应用的复杂性远超单次测试。

PostgreSQL 和 Weston 在启动过程中调用数百个 syscall,

任何一个环节的偏差都导致启动失败。

11 个独立 syscall 修复



PostgreSQL + Weston 适配

实验 4A: PostgreSQL 适配

为什么选择 PostgreSQL

PostgreSQL 在常见 Linux 应用中属于少数同时深度依赖所有核心 POSIX 子系统的类别:

- `setresuid` 切换用户身份
- `AF_UNIX socketpair` 进程间通道
- `SysV shmget` 分配共享内存
- `epoll + self-pipe` 事件循环
- `fsync` 作用于目录 `fd`
- `rename + fsync` 原子持久化

关键优势:

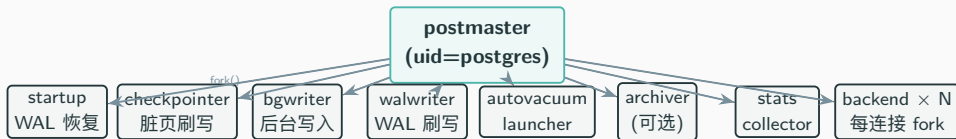
启动失败时每个步骤输出精确的错误码和系统调用名。

PostgreSQL 的启动日志本身就是最好的测试框架, 不需要 `strace`。

Web 服务器主要压力在 `epoll` 与 `sendfile`。
键值存储主要压力在 `epoll` 与内存分配。
PostgreSQL 启动过程依次经历所有核心 POSIX 子系统。

PostgreSQL 进程模型

PostgreSQL 采用每连接独立进程模型（非线程池）：



每个 fork 子进程的初始化序列：

1. `setresuid/setresgid` → 切换至 postgres 用户 (uid=70)
2. `prctl(PR_SET_PDEATHSIG, SIGTERM)` → 父进程终止时收到 SIGTERM
3. `prlimit64(RLIMIT_NOFILE, ...)` → 提高文件描述符限制
4. `getrlimit(RLIMIT_STACK)` → 计算 `max_stack_depth`

任一步骤失效均导致 worker 退出，postmaster 判定系统不稳定并中止启动。

进程间通信层

PostgreSQL 使用三条独立的 IPC 通道:

1. SysV 共享内存 (shared_buffers 页缓存层)

```
postmaster: shmget(IPC_PRIVATE, size, IPC_CREAT | 0600)
backend:    shmatt(shmid, NULL, 0) // fork 后继承映射
           shmdt(shmaddr) // 进程退出时分离
```

要求: 多进程并发 attach/detach 不可死锁; IPC_RMID 在 nattach==0 时立即销毁。

2. AF_UNIX socketpair (postmaster worker 控制通道)

```
socketpair(AF_UNIX, SOCK_STREAM) {sock0, sock1}
worker: close(sock1) // 写端关闭
postmaster: recv(sock0) // 期待返回 0 (EOF)
```

要求: 写端关闭后读端必须返回 0 (EOF) 而非 EAGAIN。

3. Self-pipe + epoll (WaitEventSet 事件循环)

```
pipe2(self_pipe, O_NONBLOCK) {rfd, wfd}
epoll_create1(EPOLLO_CLOEXEC) epfd
epoll_ctl(epfd, EPOLL_CTL_ADD, rfd, EPOLLIN)
epoll_ctl(epfd, EPOLL_CTL_MOD, latch_fd, EPOLLIN) // 反复重武装
```

要求: EPOLL_CTL_MOD 后事件不可丢失; LT 模式不可对同一 fd 返回 N 份副本。

存储层与完整启动序列

PostgreSQL 的持久化写入采用 `durable_rename` 模式:

```
write(tmp_file)           // 写入临时文件
fsync(tmp_file)           // 确保数据到达存储介质
rename(tmp_file, final)   // 原子重命名
fsync(parent_dir_fd)      // 确保目录项持久化 (POSIX 要求)
```

完整启动序列 (红色标注 = StarryOS 上曾中断的位置):

1. `setresuid/setresgid` → `uid=70` ← 空操作, 无效果
2. `socketpair(AF_UNIX, SOCK_STREAM)`
3. `bind(AF_UNIX, "/tmp/.s.PGSQL.5432")`
4. `chmod("/tmp/.s.PGSQL.5432", 0777)` ← `EPERM` (属主错误)
5. `shmget(IPC_PRIVATE, size, IPC_CREAT|0600)` ← `SMP` 死锁
6. `shmat(shmid, NULL, 0) × N` ← `SMP` 死锁
7. `epoll_ctl(epfd, EPOLL_CTL_MOD, latch_fd)` ← 事件丢失
8. `epoll_wait(epfd, events, maxevents, -1)` ← `LT` 重复事件
9. `fork()` → 子进程
10. `prctl(PR_SET_PDEATHSIG, SIGTERM)` ← 被静默忽略
11. `prlimit64(RLIMIT_NOFILE, ...)` ← 返回成功但不生效
12. `getrlimit(RLIMIT_STACK)` ← 默认 512K (应为 8M)
13. `fsync(data_dir_fd)` ← 返回 `EISDIR`
14. `rename(pg_control_tmp, pg_control)` ← 目标文件内容清零
15. `fchown(pg_control, uid=70)` ← 凭证缺失
16. `accept()` → `SIGUSR1` → `SA_RESTART` → `accept()` ← 返回 `EINTR` 而非重启
17. 并行查询 → `interrupt waker` ← 竟态, 永久挂起

修复：凭证子系统 (#246) + SA_RESTART (#247)

凭证子系统 (#246)。整个适配中工程量最大的单项修改。

```
struct Cred {  
    uid, gid      // 真实身份  
    euid, egid    // 有效身份 (权限检查)  
    suid, sgid    // 保存的 set-user-ID  
    fsuid, fsgid  // 文件系统访问专用  
    sup_groups[]  // 补充组列表  
}
```

Arc<Cred> 引用计数共享, fork 时继承。

接入点:

- faccessat2: fsuid 检查
- fchownat: CAP_CHOWN
- kill: uid/euid/suid 匹配
- IPC 所有权与权限检查
- /proc/<pid>/status 字段

setresuid(-1, 1000, -1) 的 NOCHG 语义: -1 表示该字段不修改。几乎所有 Linux daemon 依赖此细节。

SA_RESTART (#247)。

RISC-V/AArch64 上 a0/x0 的寄存器复用问题:

1. PC 回退 SYSCALL_INSN_LEN 字节 (RISC-V/AArch64=4, x86_64=2)
2. 恢复 a0 为原始参数 (而非写入 -EINTR)
3. 不调用 set_retval(0) (在 RISC-V/AArch64 上会覆盖刚恢复的 a0)

修复: 并发正确性问题 (#226, #504, #316)

IPC 共享内存死锁 (#226)。SMP=4 条件下 95% 复现率:

```
sys_shmget: SHM_MANAGER → shm_inner
sys_shmat:  shm_inner → aspace → SHM_MANAGER
             ↑           ↑
             锁顺序相反 → AB/BA 死锁
```

修复: 统一全局锁顺序为 SHM_MANAGER → shm_inner → aspace。sys_shmctl 分阶段加锁。同步修复 use-after-free 和 IPC_RMID 销毁时机。

epoll LT 就绪队列 (#504)。电平触发模式下单个就绪 fd 返回 N 份副本:

```
bug: pop_front 循环 interest 被消费 maxevents 次
fix: mem::take drain 每 interest 仅访问一次 LT 幸存者保留
```

Interrupt waker 竞态 (#316)。swap(false) 与 register(waker) 之间存在窗口。

修复: 将 register 移到 swap 之前。register-then-check 模式在本次项目中反复出现: SA_RESTART、waitpid、interrupt waker、epoll check_and_register_waker。

修复: VFS rename (#312) + EPOLL_CTL_MOD (#314)

VFS rename 原子性 (#312)。 tmpfs 上 `rename(src, dst)` 后目标文件内容全零:

```
DirNode::rename(src, dst):
    bug: forget_entry(src) 旧 DirEntry Arc 旧 page cache 释放
        lookup(dst) 新 DirEntry 空 page cache
    fix: remove_entry(src) 取出而非 forget
        insert_entry(dst, same_DirEntry) 新 page cache 保持
```

`rename` 的语义是重关联同一文件对象至新名称。对 tmpfs 此类纯内存文件系统, 维持此语义的唯一途径是保持 `DirEntry` 实例不变。

EPOLL_CTL_MOD 事件丢失 (#314)。 PostgreSQL 的 `WaitEventSet` 对 `latch fd` 反复执行 `MOD` 重武装后 `epoll_wait` 收不到事件:

```
Epoll::modify:
    1. 替换 interests[fd] 中的 Arc<EpollInterest> (旧 → 新)
    2. 旧 Arc 引用计数归零 释放
    3. ready_queue 中悬挂的 Weak<旧 Arc> upgrade() None 静默跳过
    4. 新 Arc 无路径进入 ready_queue epoll_wait 永远看不到此事件
```

修复: 替换前记录旧 `Arc` 的 `is_in_queue()` 状态; 若旧条目已在 `ready_queue` 中, 立即将新 `Arc` 的 `Weak` 推入 `ready_queue`。

PostgreSQL 适配总结

13 个 PR，按问题层次分类：

层次	PR 数	代表	特征
缺失功能	1	#246 凭证	整套子系统从头构建
语义偏差	9	#251 fsync 目录 #312 rename, #314 MOD	基本路径正确，标志位/errno/边界 与 Linux 不一致
并发正确性	3	#226 SHM 死锁	单核正确，SMP 暴露

13 个 PR 所修复的均为通用内核缺陷。凭证子系统影响所有使用 `setresuid` 的应用；`epoll LT` 修复影响 `libuv`、`Go netpoll`、`nginx`；`VFS rename` 修复影响 `SQLite` 及各类包管理器。

实验 4B: Wayland/Weston 适配

为什么选择 Weston

PostgreSQL 测试的是 POSIX 模型（进程、凭证、IPC、文件系统）。Weston 测试的是 Linux 驱动模型和用户态图形生态。两者覆盖了 StarryOS 内核接口的互补侧面。

维度	PostgreSQL	Weston
核心依赖	进程模型、凭证、VFS、IPC、epoll	驱动节点、DRM/KMS、evdev、netlink、AF_UNIX SCM_RIGHTS
测试层	syscall 语义层	驱动 ioctl + sysfs + udev 契约层
失败模式	明确错误码 + 日志	多层级初始化失败，根因深埋 5+ 层调用栈
发现方式	启动日志直接指出失败的系统调用	阅读 libudev/libinput/libdrm 源码追踪判断条件

Weston 的启动路径穿透了整个 Linux 用户态图形栈:

drm-backend (libdrm → /dev/dri/card0 → DRM/KMS) → libinput (libudev → /sys, /dev/input/eventN → evdev) → wayland-server (AF_UNIX → SCM_RIGHTS fd 传递) → event loop (epoll + timerfd)

子系统 1: DRM/KMS 图形栈: 资源枚举

设备打开与能力协商:

```
drmOpen("/dev/dri/card0") → fd  
drmSetClientCap(fd, DRM_CLIENT_CAP_UNIVERSAL_PLANES)  
drmSetClientCap(fd, DRM_CLIENT_CAP_ATOMIC)
```

libdrm 盲探约十种 capability。内核必须对未知 cap 返回 0。

KMS 资源枚举:

```
drmModeGetResources(fd)  
→ CRTC 列表      ← 显示控制器  
→ Connectors 列表 ← 物理输出端口  
→ Encoders 列表  ← 信号编码器  
→ framebuffer 列表 ← 已注册的 framebuffer  
  
drmModeGetConnector(fd, conn_id) → 显示模式列表  
drmModeGetPlane(fd, plane_id) → 像素格式 + 类型
```

属性系统: CRTC.ACTIVE 类型必须是 RANGE [0, 1]。如果是 ENUM 或 SIGNED_RANGE, weston 的 atomic backend 直接拒绝。

子系统 1: DRM/KMS 图形栈: 模式设置

Framebuffer 分配与原子提交:

```
drmModeCreateDumb(fd, w, h, bpp) ➡ {handle, size, pitch}
drmModeAddFB2(fd, w, h, format, handles[], ...) ➡ fb_id

drmModeCreatePropertyBlob(fd, &mode, sizeof(mode)) ➡ blob_id
drmModeAtomicAlloc() ➡ atomic_request

// 原子提交: CRTC + Plane + Connector 一次性生效
drmModeAtomicAddProperty(req, CRTC_ID, "ACTIVE", 1)
drmModeAtomicAddProperty(req, CRTC_ID, "MODE_ID", blob_id)
drmModeAtomicAddProperty(req, PLANE_ID, "FB_ID", fb_id)
drmModeAtomicAddProperty(req, PLANE_ID, "CRTC_ID", crtc_id)
drmModeAtomicCommit(req,
    DRM_MODE_ATOMIC_ALLOW_MODESET | DRM_MODE_PAGE_FLIP_EVENT)
```

MODE_ID blob 回环: 用户态序列化 → 内核原样保存 → 用户态读回完全一致的字节序列。

PAGE_FLIP 事件: commit → 硬件翻页 → 内核写入 `drm_event_vblank` → `epoll` 唤醒 → 提交下一帧。

Mode list 使用 CVT-RBv1 真实时序参数合成。随便填的数字会被 `weston mode validator` 拒绝。

子系统 2: evdev 输入 + libinput

libinput 是 Weston 的输入后端，通过 libudev 发现设备、通过 evdev ioctl 查询能力。

设备发现:

```
udev_enumerate_add_match_subsystem("input")
udev_enumerate_scan_devices()
➤ 遍历 /sys/class/input/
➤ realpath() 解析软链 ➤ dirname() 定位 ➤ 读 uevent
➤ 遍历 /sys/dev/char/<major>: <minor>
➤ 检查 /run/udev/data/c<major>:<minor> ➤ 判定是否已初始化
```

设备能力查询:

```
ioctl(fd, EVIOCGBIT(EV_KEY, sizeof(keys)), &bits) ➤ 按键能力位图
ioctl(fd, EVIOCGBIT(EV_ABS, sizeof(abs)), &bits) ➤ 绝对轴能力
ioctl(fd, EVIOCGPROP(0), &prop_bits) ➤ 设备属性位
ioctl(fd, EVIOCGABS(ABS_X), &abs_info) ➤ {min, max, fuzz, flat}
```

libinput 分类决策树: INPUT_PROP_POINTER + BUTTONPAD → 触控板; INPUT_PROP_DIRECT + ABS_X/Y → 触屏; 无属性 + 按键 → 键盘。

缺失任何一个 ioctl 返回值，设备无法被正确分类，libinput 将其事件当作噪声丢弃。

子系统 3: Wayland 协议与 FD 传递

Wayland 协议的 IPC 机制:

```
// 客户端
memfd_create("wayland-shm", MFD_ALLOW_SEALING) ❷ shm_fd
ftruncate(shm_fd, w * h * 4)
mmap(NULL, size, PROT_WRITE, MAP_SHARED, shm_fd, 0) ❸ 像素 buffer
write(pixels)
sendmsg(sock, {SCM_RIGHTS, [shm_fd]}) // 传递 fd 给合成器

// 合成器 (Weston)
recvmsg(sock, MSG_DONTWAIT, {SCM_RIGHTS}) ❹ 收到 shm_fd
mmap(NULL, size, PROT_READ, MAP_SHARED, shm_fd, 0) ❺ 读取像素
```

SCM_RIGHTS 在 SOCK_STREAM 上的 byte-mark 语义:

一次 `sendmsg` 中, `cmsg` 携带的 `fd` 与同一调用写出的第一个字节原子交付。接收端 `recvmsg` 在读到此 `fd` 对应的字节范围的首字节时同时获得 `fd`。

Linux 实现: 将 `[start_byte, end_byte]` 与 `scm_fp_list` 绑定; `recvmsg` 截断读取在下一条 `cmsg` 的 `end_byte` 处。

消息边界不匹配 → `fd` 与合成器期望的 `fd` 错位 → Wayland 协议判定违规并断开连接。

子系统 4: memfd 与 F_SEAL_*

memfd 是 Wayland 共享内存机制的内核基础:

```
memfd_create("wayland-shm", MFD_ALLOW_SEALING) & fd
ftruncate(fd, size)
fcntl(fd, F_ADD_SEALS, F_SEAL_SHRINK) // 禁止缩小
fcntl(fd, F_ADD_SEALS, F_SEAL_GROW)   // 禁止扩大
fcntl(fd, F_ADD_SEALS, F_SEAL_WRITE)   // 禁止写入
fcntl(fd, F_ADD_SEALS, F_SEAL_SEAL)    // 不可逆锁定
```

Seal 语义渗透到三个独立内核入口:

Seal	影响的系统调用	行为
F_SEAL_SHRINK	ftruncate new_size < old_size	拒绝, 返回 EPERM
F_SEAL_GROW	ftruncate new_size > old_size; write 跨 EOF	拒绝扩大; write 被拒且文件大小回退
F_SEAL_WRITE	write; mmap(PROT_WRITE, MAP_SHARED)	拒绝写入; mmap 返回 EPERM
F_SEAL_SEAL	fcntl(F_ADD_SEALS)	拒绝所有后续 seal 修改

并发安全: add_seals 使用 compare_exchange 循环原子合并; set_len_sealed 在持锁状态下完成"读取 → 校验 → 修改"三步。

隐性契约: sysfs、udev 与设备发现

Weston 适配中最困难的部分。三个 bug 都不是 syscall 层面的问题:

1. sysfs class 目录必须是软链:

`/sys/class/drm/card0` → 必须软链到 `/sys/devices/virtual/drm/card0`

libudev: `realpath()` → `dirname()` → 读 `uevent`

普通目录: `dirname("/sys/class/drm/card0") = "/sys/class/drm"` (无 `uevent`)

2. evdev minor 号必须从 64 起:

Linux: `EVDEV_MINOR_BASE = 64` → `/dev/input/event0` → `minor=64`

StarryOS 原: sysfs 写 "13:64" 但设备节点实际为 "13:1" (`minor = i+1`)

libinput: `fstat()` → `st_rdev = makedev(13, 1)` → 找不到匹配 → 报错

3. /run/udev/data/ 预填空文件:

libudev: `access("/run/udev/data/c13:64", F_OK) == 0` → "initialized"

StarryOS: 无 `udev`d → 文件不存在 → libinput 判定设备未初始化 → 跳过

这三种契约散布在 udev 规则和用户态库源码中, 没有任何集中文档记载。

修复: DRM/KMS 设备节点 (#506)

Weston 首个阻塞点: `drmOpen("/dev/dri/card0") → ENOENT`。

从零创建 DRM 设备节点, 实现 legacy 与 atomic 两条 KMS 路径:

路径	ioctl	用途
Legacy	SETCRTC	基本模式设置
	PAGE_FLIP	翻页通知
	WAIT_VBLANK	等待垂直消隐
Atomic	ATOMIC_COMMIT	原子提交 CRTC+Plane+Connector
	ATOMIC_TEST_ONLY	测试模式
通用	GETRESOURCES, GETCONNECTOR, GETPLANE	资源枚举
	GETPROPERTY, GETPROPBLOB, CREATEPROPBLOB	属性系统

关键细节: 未知 capability 返回 0 (libdrm 探针); CRTC.ACTIVE 类型为 RANGE [0, 1] (weston 硬编码检查); MODE_ID blob 字节级回环。

测试: 83 项断言覆盖 version / modeset / atomic 三条路径, 4 架构 PASS。

修复: evdev 多设备与输入 ioctl (#513)

设备分类误判。

QEMU virtio-keyboard 因 BTN_MISC 在同一字节报位, 键盘误判为鼠标。后注册设备覆盖先注册的, event0 消失。

修复: 每个设备均暴露为 /dev/input/eventN, 首个鼠标 alias 到 mice。

EVIOCGPROP 未实现。

libinput 通过此 ioctl 获知设备属性位 (INPUT_PROP_POINTER、INPUT_PROP_DIRECT、INPUT_PROP_BUTTONPAD)。未实现时所有位为 0, libinput 判定“无 pointer 属性”→ 将触屏事件当作噪声丢弃。

EVIOCGABS 未实现。

libinput 查询 ABS_X.min/max、ABS_Y.min/max 获取坐标范围。全零时触屏区域为 0×0, 所有事件在边界外。

修复: axdriver_input_trait 新增 get_prop_bits() 和 get_abs_info() 默认方法; virtio-input 驱动实现透传。

修复: Weston bringup 综合修复包 (#509)

Wayland 路径上最复杂的单项修改, 含 7 项互相依赖的修复:

编号	修复	问题
N1	mmap PROT_WRITE 自动补 PROT_READ	RISC-V 特权规范 R=0 W=1 为 reserved PTE
N2	EventDev::register 不 再无条件 wake	LT 下死循环 500Hz
N3	/dev/input 设备分类 修正	BTN_MISC 导致键盘误判为鼠标
P1	virtio-input IRQ 唤醒	仅靠 16ms tick 兜底, 输入延迟 16ms
P2	register_irq_waker 重 构	IRQ 上下文 SpinNolrq 可能死锁
P3	EventDev 多事件缓冲 (VecDeque, 256)	单槽位 read_ahead, burst 丢事件
P4	AF_UNIX SCM_RIGHTS + MSG_DONTWAIT	cmsg 被忽略, fd 凭空消失

P4 实现:

```
PendingCmsg { start_byte, end_byte, cmsg } 从列  
send: [start_byte, end_byte] 与 cmsg 绑定入队  
recv: 读取截断在下一条 cmsg 的 end_byte 处  
cmsg 与对应字节范围的首字节原子交付
```

register-then-check 模式在 IRQ waker 中再次应用: 先 register(waker), 再 swap(false) 检查。

修复: aarch64 架构支持 (#510, #511)

#510: EL0 cache 指令使能。 aarch64 上 Weston 在 `__libc_start_main` 之前收到 SIGTRAP。

根因: SCTLR_EL1 寄存器三位未置位:

位	名称	控制
bit 15	UCT	EL0 访问 CTR_EL0 (cache type register)
bit 14	DZE	EL0 执行 DC ZVA (data cache zero by address)
bit 26	UCI	EL0 执行 DC CVAU 和 IC IVAU (cache maintenance)

musl/glibc 启动早期需读 CTR_EL0 获取 cache line size; Mesa 的 SIMD 路径需 DC ZVA 零拷贝。

#511: GICv3 + CNTV for Apple HVF。

Apple Hypervisor Framework 下 GICv2 路径访问 GICC MMIO 时 HVF 无法提供 ESR.ISV → QEMU `assert(isv)` 失败。CNTV 被 EL2 占用。

可用路径: GICv3 system-register CPU interface + CNTV virtual timer。修复增加 `gic-v3` 和 `cntv-timer` 两个 feature flag。默认 QEMU TCG 与物理板保持 GICv2 + CNTV 路径不变。

Weston 适配总结

8 个 PR，按子系统分层推进:

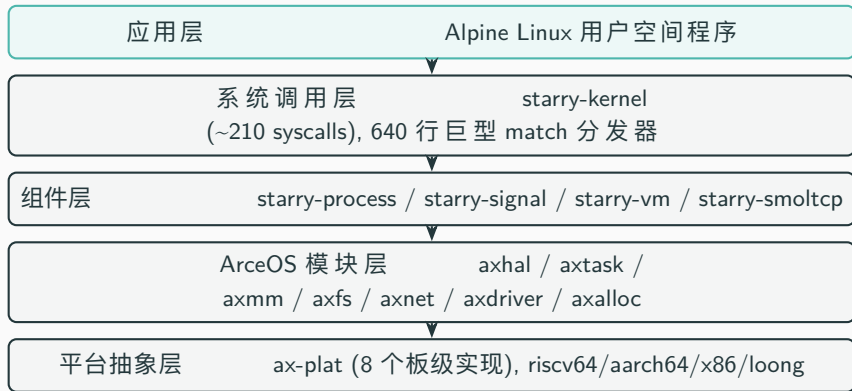
子系统层	PR	修复内容
架构基础	#510, #511	aarch64 EL0 cache 指令、GICv3+CNTV
设备节点	#506	/dev/dri/card0: DRM/KMS legacy+atomic 双路径
sysfs/udev	#508	class 软链、evdev minor=64、/run/udev/data/ 预填空
输入	#513	每设备 eventN、EVIOCGPROP/EVIOCGABS
图形协议	#507	memfd F_SEAL_SHRINK/GROW/WRITE/SEAL + 并发安全
网络	#512	NETLINK_ROUTE/GENERIC/KOBJECT_UEVENT + rtnetlink 应答
综合修复	#509	PROT_WRITE、IRQ waker、SCM_RIGHTS byte-mark、MSG_DONTWAIT

内核图形栈的兼容性不是 checklist 式的功能对照。它是对 Linux 用户态生态中 [隐性契约的逐层发现](#)。修复本身加起来不过数百行，但定位问题需要穿透五层抽象（Weston → libinput/libdrm → libudev → sysfs/evdev → 内核），每层都可能隐藏一个微小的格式偏差。

StarryOS 架构分析

架构概览

StarryOS 在 ArceOS unikernel 的模块体系之上叠加一层 Linux syscall ABI 兼容层:



核心设计: 复用 ArceOS 的硬件抽象、驱动框架、调度器、页表、文件系统, 通过约 640 行的巨型 match 分发器实现约 210 个 Linux syscall 语义。StarryOS 代码量约 60% 在 syscall/ 及其子模块中。

有利于 AI 开发的架构特征

1. **分层模块化。**修改 `starry-signal` 无须理解 ArceOS 的 `virtio` 驱动。`starry-process` 作为纯数据模型，不依赖调度器与 MMU。AI 代理可仅阅读相关层源码，不必加载整个内核上下文。
2. **类型系统的编译时验证。**四种内存映射后端通过 `enum_dispatch` 统一为 `Backend` 枚举，编译器强制穷举所有变体。新增变体时编译器自动标记所有未处理分支。
3. **确定性的信号投递。**信号在 `syscall`/异常/缺页返回后同步检查，非异步抢占。信号投递时机完全由 `syscall` 状态驱动，可复现、可测试。
4. **资源作用域的可追踪性。**`scope_local` 机制将 FD 表和 FS 上下文绑定到进程作用域。每个 fd 必然属于某个活跃作用域，不存在悬空引用。
5. **Linux 行为预言机。**`starry-harness` 在 Docker Linux 上运行与 StarryOS 相同的测试输入，逐字段对比。测试框架本身编码了正确性标准。

产生系统性摩擦的架构特征

1. **巨型 match 分发器。**约 210 个 syscall 集中在一个函数中，syscall 之间没有模块边界。验证一个 syscall 修复需”构建内核 → 制作 rootfs → 启动 QEMU → 运行测试”，单次迭代 3-5 分钟。
2. **能力发现机制缺失。**8 个 syscall 返回空操作的匿名 fd (userfaultfd、io_uring_setup、memfd_secret 等)，调用成功但无实际功能。对比: fanotify_init 返回 -ENOSYS 反而提供了明确信号。
3. **层间隐式耦合。**TaskExt::on_enter/on_leave 通过 scope_local 隐式副作用传递进程状态，无显式返回值。
4. **特性标志透明性缺失。**三层特性级联，特性冲突不在编译时报错。plat-dyn 在 x86_64/loongarch64 上被静默禁用以回退到静态平台模式。
5. **凭证模型的语义偏差。**credentials 存储在 Thread.cred (per-thread)，Linux 为 per-process。set_cred() 遍历所有线程同步凭证时存在时间窗口。

执行 vs 决策: 六周数据

32 个 PR 的完成过程提供了关于 AI 辅助开发定位的具体数据:

维度	AI	人工	说明
编写代码	~70%	~30%	模板代码、测试生成、配置修改
生成测试	~80%	~20%	C 测试程序自动化
运行 clippy/fmt	~95%	~5%	完全自动化
判断根因	~10%	~90%	需理解子系统交互与语义
确定修复方向	~10%	~90%	在多个方案中选择
选择锁策略	~5%	~95%	传递性死锁检查不可靠
评估传递性影响	~5%	~95%	修改对全局状态的影响
架构决策	0%	100%	如凭证存储 per-thread vs per-process

AI 承担了约 70% 的执行工作量, 但约 90% 的决策工作量依赖人工判断。AI 的核心价值在于消除” 已知问题所在但修改成本过高” 的摩擦, 而非替代” 问题尚不明确需要探索” 的认知过程。

重新设计：六个方向

若重新设计一个面向 AI 辅助开发友好的内核，核心思路是使架构内建更多“变更可被验证”的机制：

改进	当前	目标
可插拔 syscall 注册表	640 行 match, 无模块边界	每个 syscall 声明编号 + 签名 + 返回类型, 独立编译、测试、fuzz
显式能力矩阵	8 个 syscall 返回静默成功的 dummy fd	/proc/sys/kernel/capabilities 暴露实现状态
层级间形式化契约	TaskExt 依赖隐式副作用	on_enter() → ActiveScope、on_leave(scope) 显式返回值
规约驱动开发	无机器可读 syscall 语义规约	YAML/TOML 规约驱动测试生成和 fuzz
自动化差异测试	Linux 对照需手动操作	每个 PR 自动 Docker Linux 上运行相同输入
验证基础设施统一	上游 CI 与 monorepo CI 测试集不重叠	一次 CI 运行覆盖所有相关测试

方向是在现有层次之间插入显式的契约和验证点，而非推倒重来。

反思 + OS 课程建议

对兼容性的重新理解

六周的工作改变了对“操作系统兼容性”的理解。

此前认为兼容性即实现 syscall 表: 对照 Linux 的 syscall 编号列表逐一实现, 跑通即完成。

实际情况:

兼容性的真正难点不在系统调用的存在性, 而在每个系统调用的精确语义、每个错误条件的 `errno` 返回值、每个边界情况的处理方式。

- `setresuid(-1, 1000, -1)` 的 `NOCHG` 语义
- `prlimit64` 返回 `Ok(0)` 但静默不生效
- `fsync` 作用于目录 `fd` 返回 `EISDIR` 还是 `Ok(0)`
- `epoll_pwait` `sigsetsize=16` 返回 `EINVAL` 还是接受

用户态生态中的隐性契约同样关键: `/sys/class/drm/card0` 必须是软链, `evdev` `minor` 号必须从 64 起, `/run/udev/data/` 必须预填空文件。这些都不对应任何一个缺失的系统调用, 但任何一个偏差都导致应用静默失败。

Linux 兼容性本质上是“成千上万个微小 ABI 的正确实现”, 而非“约 400 个系统调用的存在性检查”。

从这次实验看 OS 教学

不写通用建议。只写这次实验让我意识到的、在课本和讲义里学不到的三件事。

修内核与写内核是两种不同的技能。

写内核是自上而下：你决定设计，你定义接口，你控制复杂度。修内核是自下而上：设计已经定了（Linux），接口已经定了（210 个 syscall + 数百个 ioctl），复杂度已经在了（15 年用户态生态的累积）。你没有控制的奢侈，你只能理解。

课程前三年教的全是“怎么写”。这次六周全在“怎么修”。修的技能与写的技能有本质区别。修需要读 man page 找边界条件、写最小复现用例、对照 Linux 确认预期行为、最小改动修复。这组方法在现有课程中未被作为独立技能教授。

最难的 bug 源于用户态代码的假设从未被记录，而非内核代码本身写错。

`/sys/class/drm/card0` 必须是软链。evdev minor 号必须从 64 起。`/run/udev/data/` 下必须有空文件。这三项分别花了数小时在“找”上，不在“修”上。

真实应用是更好的测试框架。

PostgreSQL 在 initdb 过程中测试了 fsync 对目录 fd 的行为、setresuid 的 NOCHG 语义、rename 后 page cache 保持、epoll CTL_MOD 事件保持。没有任何单元测试覆盖这些组合。

真实应用不知道你的内核实现细节，它只会用它认为理所当然的方式调用系统调用。这种“无知”使它成为最诚实的测试者。

最终产出

最终产出汇总

实验 1: AI 流水线

- starry-harness 插件
- 9 技能 + 3 代理 + 16 脚本
- 支撑全部 32 个 PR

实验 4B: Wayland/Weston

- 8 个 Wayland 专属 PR
- DRM/KMS 双路径
- sysfs 软链 + SCM_RIGHTS
- Weston 14 帧渲染

实验 2: syscall 修复

- 11 个独立 PR
- timerfd (fd 完整实现)
- preadv/pwritev (ABI)
- mremap (内存管理)

架构分析

- 5 个有利于 AI 的特征
- 5 个产生摩擦的特征
- 6 个重新设计方向

实验 4A: PostgreSQL

- 13 个 PG 发现 PR
- 凭证子系统、SA_RESTART
- IPC 死锁、VFS rename
- postmaster + initdb + 基本查询

32 个已合入 PR
30+ 回归测试用例
4 架构覆盖

致谢

感谢评审老师的宝贵时间

欢迎提问与讨论

Joseph Joshua (jsph273) · rcore-os/tgskits · 2026-05-30